

✓ Toms_Supervised_HPC_Assignment

Full-Stack Supervised Learning, Infrastructure Profiling & Case Study

Welcome to Assignment 4! This week, we are bridging the gap between low-level hardware orchestration and high-level machine learning strategy.

- **Part 1 (Coding & Infrastructure Benchmarking):** You will build an optimized, loop-free K-Nearest Neighbor pipeline using **JAX** to analyze telecom customer churn. You will configure and dispatch this workload across our cluster's **CPU, GPU, and TPU** pools to benchmark execution performance deltas.
- **Part 2 (Business Case Study):** You will analyze real-world system constraints for a vacation rental provider to make architecture and engineering recommendations to a CEO.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
-----
---
MessageError                                Traceback (most recent call
last)
/tmp/ipykernel_722/1408506528.py in <cell line: 0>()
      1 from google.colab import drive
----> 2 drive.mount('/content/drive')

-----
      3 frames -----
/usr/local/lib/python3.12/dist-packages/google/colab/message.py in
read_reply_from_input(message_id, timeout_sec)
    101 ):
--> 102     if 'error' in reply:
    103         raise MessageError(reply['error'])
    104     return reply.get('data', None)
    105
```

Next steps: [Explain error](#)

✓ Part 1: Hands-On Infrastructure & Core Coding

✓ Setup

First, let's import our processing libraries and inspect the current cluster topography configured for your student account environment.

```
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import jax
import jax.numpy as jnp
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

%matplotlib inline

print("=== Cluster Topology Diagnostics ===")
print(f"All visible hardware devices: {jax.devices()}")
for backend in ['cpu', 'gpu', 'tpu']:
    try:
        devices = jax.devices(backend)
        print(f"  Backend '{backend}': {len(devices)} active execution pool slice(s)")
    except RuntimeError:
        print(f"  Backend '{backend}': 0 (Unconfigured / offline on this partition node)")
```

Exercise 1

Load the CSV file named `Churn.csv` into a Pandas DataFrame assigned to a variable called `df`. Display the first 10 rows of the dataset to verify structure.

```
# Write your Exercise 1 code below:
import pandas as pd
df=pd.read_csv('/content/sample_data/Churn_Dataset.csv')
print(df.head(10))
```

Exercise 2

Which specific column indicates whether a user has churned? How many total users are present in your dataset?

```
# Write your Exercise 2 code below:
print("Churn column name:", 'Churned')
print("Total users:", df.shape[0])
```

Exercise 3

Use `df.describe()` to explore the data distribution across your columns. Why are the data row counts inconsistent across variables instead of matching the full 5,000 dataset boundaries identically?

```
# Write your Exercise 3 code below:
print(df.describe())
# the command only counts non-null values. If a column is missing, it will lower the number of rows
```

Exercise 4

Impute and fill all missing vector fields with the calculated median value of their respective datasets. Verify that your adjusted row count boundaries reflect a clean 5,000 across all dimensions.

```
# Write your Exercise 4 code below:
df= df.fillna(df.median(numeric_only=True))
print(df.count)
```

Exercise 5

Separate your input data boundaries into structural features and classification labels. Assign your clean feature array to the matrix variable `X` and labels to the variable `y`.

```
# Write your Exercise 5 code below:
x= df.drop('churned',axis=1)
y= df['churned']
```

Exercise 6

Split your dataset parameters into a 70% training configuration and a 30% testing configuration. Assign the resulting metrics to variables named `X_train`, `X_test`, `y_train`, and `y_test` respectively.

```
# Write your Exercise 6 code below:
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test= train_test_split(x,y,test_size=0.3,random_state=42)
```

Exercise 7: JAX Hardware Target Allocation

To enable explicit execution of our machine learning arrays on specialized hardware, complete the helper function below. Use `jax.device_put()` to manually allocate your NumPy datasets into memory arrays assigned to a designated cluster target engine backend (`'cpu'`, `'gpu'`, or `'tpu'`).

```
def allocate_workload_to_hardware(X_train_np, X_test_np, backend_target='cpu'):  
    """  
    Explicitly captures a target cluster device handle and assigns arrays to memory storage limits.  
    """  
    # 1. Fetch the target device primitive from your active backend topography  
    device = jax.devices(backend_target)[0]  
  
    # 2. Push inputs to device memory  
    # Hint: X_train_jax = jax.device_put(your_numpy_array, device=device)  
    X_train_jax = jax.device_put(X_train_np, device=device)  
    X_test_jax = jax.device_put(X_test_np, device=device)  
  
    return X_train_jax, X_test_jax
```

Exercise 8: Vectorized XLA Distance Kernel

Accelerators like GPUs and TPUs perform poorly with loop iterations. Write a purely vectorized, loop-free mathematical calculation to resolve standard pairwise Euclidean distances using JAX numpy primitives (`jnp`).

Decorate your function with `@jax.jit` to instruct the hardware-aware XLA compiler to map your algorithmic execution paths down into bare-metal accelerator machine instructions.

```
@jax.jit  
def compute_cluster_pairwise_distances(X_train_jax, X_test_jax):  
    """  
    Computes Euclidean distances between matrix variables of test samples and train samples.  
    Returns array matrix dimensions: (n_test_samples, n_train_samples)  
    """  
    # Hint: Use matrix transformations to map  $(A - B)^2 = A^2 - 2AB + B^2$  directly inside jnp primitives  
    test_sq=jnp.sum(X_test_jax ** 2, axis=1, keepdims=True)  
    train_sq=jnp.sum(X_train_jax ** 2, axis=1)  
    distances = jnp.sqrt(  
        test_sq - 2 * jnp.dot(X_test_jax, X_train_jax.T) + train_sq  
    )  
    return distances
```

Exercise 9: Nearest Neighbors Prediction Pipeline

Extract the closest 5 neighbor indices from your computed distance matrix calculations, map them back to their ground-truth index assignments in `y_train`, and compute the majority class designation to yield your predictions array (`y_pred`).

```
def predict_labels_knn(distance_matrix, y_train, k=5):  
    # Process array entries to isolate top-k shortest distance paths and execute vote functions  
    y_train_jnp = jnp.asarray(y_train) # Convert y_train to JAX array for compatible indexing  
    nearest_idx=jnp.argsort(distance_matrix,axis=1)[:,:k]  
    neighbor_labels=y_train_jnp[nearest_idx].astype(int) # Index JAX array with JAX array  
  
    y_pred = jnp.array ([  
        jnp.argmax(jnp.bincount(labels_row, minlength=2))  
        for labels_row in neighbor_labels  
    ])  
    return y_pred
```

Exercise 10

Run your prediction pipeline on your testing datasets, and use `accuracy_score` to determine how accurate your model is at identifying churn behavior.

```
# Write your Exercise 10 model evaluation below:
X_train_jax, X_test_jax = allocate_workload_to_hardware(X_train.to_numpy(), X_test.to_numpy(), backend_target='cpu')

distance_matrix=compute_cluster_pairwise_distances(
    X_train_jax,
    X_test_jax
)

y_pred=predict_labels_knn(distance_matrix, y_train, k=5)
accuracy=accuracy_score(y_test,np.array(y_pred))
print("Accuracy:", accuracy)
```

✎ Exercise 11: Cluster Profiling & Performance Delta Log

Execute the profiling code sequence below. It will cycle your compiled kernel operations automatically across the available CPU, GPU, and TPU nodes in our cluster configuration.

Review your logged output table metrics and answer the written prompt detailing the execution latency changes or constraints you notice across different processing architectures at this dataset footprint scale.

```
cluster_backends = ['cpu', 'gpu', 'tpu']
benchmark_metrics = {}

for backend in cluster_backends:
    try:
        # Allocate variables into localized environment channels
        X_tr_jax, X_te_jax = allocate_workload_to_hardware(X_train.to_numpy(), X_test.to_numpy(), backend_target=backend)

        # Execution warm-up cycle to decouple XLA compile overhead from calculation throughput times
        _ = compute_cluster_pairwise_distances(X_tr_jax, X_te_jax).block_until_ready()

        # Precise hardware telemetry capture
        start_mark = time.perf_counter()
        distances = compute_cluster_pairwise_distances(X_tr_jax, X_te_jax).block_until_ready()
        end_mark = time.perf_counter()

        elapsed_time_ms = (end_mark - start_mark) * 1000
        benchmark_metrics[backend] = f"{elapsed_time_ms:.4f} ms"

    except (RuntimeError, ValueError) as e:
        benchmark_metrics[backend] = "Target hardware cluster pool unconfigured or busy"

print("\n=== Cluster Performance Delta Report ===")
for device_type, execution_speed in benchmark_metrics.items():
    print(f"Hardware Layer: {device_type.upper():<4} | Kernel Compute Latency: {execution_speed}")
```

Written Response: Student Performance Observations

Prompt: Document and analyze the performance numbers you captured above. Explain what architectural features or hardware bottlenecks (e.g., memory transfers, low-precision advantages, small matrix sizes) account for the changes in speed across CPU, GPU, and TPU execution targets.

Your Answer Here: The model had an accuracy of 86%, meaning it did a good job of classifying the test samples. The CPU completed the kernel in 7 ms, while the GPU and TPU were unavailable due to hardware clusters that were busy or unconfigured. Usually though, GPU's and TPU's outperform CPU's by having a more complex neural network. In this experiment, CPU had a strong and reliable answer and had a strong accuracy

✎ Part 2: Case Study – Real-World ML Systems Strategy

✎ Operational Scenario Context:

You are an engineering manager at **"Awesome Homes,"** a real estate enterprise specializing in vacation property rentals. Over the last five consecutive years, overall sales have declined by 10% annually. Market user reviews consistently indicate that your internal retail price catalog items are perceived as either severely inflated or drastically under-priced relative to true market conditions.

Historically, property valuations at Awesome Homes are set through legacy processes where teams of localized real estate agents manually trace arbitrary indicators and rely on subjective experience to fix rental metrics. Your CEO has officially appointed you to direct a new cross-functional systems team tasked with implementing modern machine learning frameworks to dramatically improve rental valuation estimation accuracy at scale.

Provide clear, comprehensive engineering answers to the following system deployment questions based on this strategy scenario:

Question 1

Many of the manual real estate agents at Awesome Homes, who are also close personal friends of yours, are highly concerned about being rendered obsolete or replaced entirely by automated software architectures. What strategic steps can you introduce to gain their buy-in and coordinate a productive roadmap for this new AI initiative?

Your Answer Here: I would make this initiative as augmenting agents instead of replacing them, and backing it with commitments. I would involve agents early as domain experts on the problem which could push the need for seasonal demand. Secondly, I would pose the AI as an estimate for properties needing human review, which would limit visits and improve relationships. Most importantly, I would have one on one conversations about how their roles will evolve, what new skills will be valued, and timeline and commitments the company can reasonably handle. Joining the technical route with genuine care turns an attack to a resolution.

Question 2

Is this automated property estimation initiative structurally classified as a **Supervised** or **Unsupervised** machine learning problem? Detail the reasoning behind your choice.

**Your Answer Here:* This is a Supervised Learning problem. Awesome Homes has data where each property with features paired with actual outcomes. Because we have that data, we can train a model to learn the mapping from property features to price, then predict prices for each property. This is not unsupervised learning, since we have a definite value that the trained model is trying to predict.

Question 3

What core categories of data, historical fields, and training parameters do you need to compile to build a robust model pipeline?

Your Answer Here: Property Features: square footage, bed/bathroom, property size, property age, and more. LOCATION: proximity to beaches or significant landmarks. Historical prices: past prices, booking frequency, seasonal prices Reputation: guesting rating and reviews

Question 4

Your infrastructure engineers bring forward several variations of machine learning models for estimating rental values. How do you properly organize your training cycles, cross-validation configurations, and validation testing splits to evaluate them fairly?

Your Answer Here: I'd mainly put it into training as that would be the most important, but also validation with the actual prices and adjusting the most and repeating training if needed to simulate real deployment conditions where the model predicts future prices. I'd also cross-validate on the training set to make sure performance isn't dependent on one lucky data point, remodeling the hyperparameters. Finally, I'd also train across seasons to train the model on each season and the varied price related to each season, preventing the model from appearing strong overall but also lacking in important sub segments.

Question 5

By what mathematical or statistical metrics will you evaluate the quality and precision of your models' continuous price prediction metrics?

Your Answer Here: I would mainly adapt the model by comparing the R(squared) value with the validation set and the output of the model. This is fundamentally the accuracy of the model and how well it is performing. Also the MAE and the RMSE which is the Mean Absolute Error and the Root Mean Squared Error. Both of these basically have the difference between the real value and the predicted value to tell how far off the model is. The only difference is the RMSE punishes the difference more. I would track all of them as individual results of each can be misleading

Question 6

How will you scientifically determine whether your automated model inferences are yielding significantly better, more profitable property valuations than the legacy baseline results generated by traditional manual real estate agent assessments?

Your Answer Here: I'd run a controlled comparison using data where both agent valuations and actual realized outcomes are known. I'd first back track the model against listings that were already priced, comparing the models predictions to what actually was listed and evaluating the error. Beyond the model, I'd also look at business success to evaluate if the model's success was just luck or true improvement. Finally, I'd pilot the model on a subset of properties, letting me compare real-world revenue and occupancy outcomes directly before the release of the model.

Question 7

During early validation, you run one of your candidate configurations over your localized training dataset entries. The model's valuation metrics return incredibly sound, consistent performance metrics internally. However, the moment your deployment layer introduces entirely new, unseen real-world market datasets, prediction quality falls apart completely.

Explain what structural issue is occurring inside this model pipeline architecture.

Your Answer Here: This is a case of overfitting. The model has learned the training data too precisely instead of identifying the proper pattern between property features and market value. As a result, the data seems to be completed rather than generalizing against new market conditions. This usually happens when the model is too complex relative to the training data, when the data is inaccurate to the real market, or when validation wasn't properly held out from training during the model development, giving false or overconfident results.

Question 8

What concrete technical steps, algorithmic adjustments, or data pipeline modifications can you enforce to mitigate this performance collapse and enhance general validation stability?

Your Answer Here: Increase and diversify training data meaning pull in listings from diverse seasons, regions, and property types so the model sees the broader market and thus will adjust to it. Regularization which would apply L1/L2 regularization to penalize complex models and discourage the model from fitting noise. The most simple is to simplify the model complexity to generalize the model often making it more rounded for the general market. Lastly, but most important is to stop the model if the validation loss is too high. This ensures the model doesn't memorize training-specific noise.

Question 9

Your engineering core compiles three distinct production model candidates whose accuracy success parameters are virtually identical. The specific lifecycle metrics match the following matrix properties:

Model Profile	Compute Time to Train	Prediction Speed per Input	Average Prediction Error Metric
Model A	24 Hours	≤ 1 Second	5% Error
Model B	6 Hours	6 Hours	1% Error
Model C	5 Minutes	2 Minutes	10% Error

Which configuration path will you officially present to your CEO for production roll-out? Justify your choice based on execution runtime constraints, corporate financial impacts, and deployment trade-offs.

Your Answer Here: For this situation, I would recommend Model A. While Model B has the lowest error of 1%, its 6 hr prediction time is not practical in any scenario. And in our scenario, booking requests need responses quickly and the 6 hr per prediction is unusable at business speed, regardless of accuracy. Model C is fast and almost trains instantly, but 10% is too imprecise for a pricing system meant to correct the current problem of our customers. Model A is the best of both medians having a small prediction time, a decent training time and a 5% error which could be further trained with more data to improve accuracy. With that said, I'd present this model to the CEO emphasizing that this is the only model that meets the operational speeds necessary for business while aiming for accuracy.